Effect Boundary Lab

RESEARCH DISCUSSION BRIEF

Vinay Muppalla | Independent systems study | 23 October 2026

Question. When can a restarted worker safely repeat an external operation after the destination's duplicate-protection record expires?

Why it matters. Backend jobs, data ingestion and agent tools often mutate a second system. A local transaction cannot commit an external HTTP action and its own receipt atomically. Retrying can duplicate work; stopping can leave work undone.

Artifact. A dependency-free Python lab with a loopback HTTP destination, separate SQLite stores, real child-process exits, four policies, eight scripted scenarios, three synthetic payload variants, raw per-trial evidence and a separate verifier. The variants share the same append-effect semantics; this is not a three-provider field trial.

Observed results. 1,152 factorial trials, 1,008 injected worker exits. Destination keys prevented in-window duplicates but produced 36 excess effects after receipt expiry. Guarded retry produced zero excess effects and zero false confirmations across 288 trials, while quarantining 72 operations; 36 quarantined operations had no remote effect. These are bounded deterministic outcomes, not production reliability rates.

Distinctive evidence. A paired witness creates identical durable client intent records with zero versus one remote effect. Without additional destination evidence, automatically retrying or declaring success can both be unsafe in one history.

Contribution and limits. The integrated benchmark and evidence package are original to this artifact. Keys, receipts and quarantine are established. The fixture uses virtual time, one surviving provider controller, synthetic append operations and sequential clients. No novel exactly-once algorithm, deployment impact or peer-reviewed publication is claimed.

Research interests. Reliable data systems, empirical software testing, durable workflow execution and trustworthy agent infrastructure. Next work: authenticated destination reconciliation, longer fault schedules and independent replication.

For employers. Demonstrates Python, SQL transactions, HTTP contracts, subprocess fault injection, concurrency testing, evidence validation and clear technical communication.

Explore. https://vinaychowdary9946-blip.github.io/research/

Contact. mvinay988579@gmail.com